MOONVEIL

Sistema de herramientas e interfaz de la barra



Proyecto: Moonveil (Yellow Stone) · Unity 6000.6.0f1
Módulo: 2º DAM · Primer trimestre
Fecha: 26 de octubre de 2026

Índice

1. Qué se ha hecho (resumen)
2. Cómo se juega: controles y herramientas
3. Archivos nuevos y modificados
4. Cómo encajan las piezas (esquema)
5. Explicación de los scripts nuevos, uno a uno
6. Cambios en los scripts que ya existían
7. Los gráficos: iconos e interfaz en pixel art
8. La interfaz (barra de herramientas)
9. Problemas que surgieron y cómo se resolvieron
10. Cómo probarlo
11. Posibles mejoras
12. Glosario de conceptos

1. Qué se ha hecho (resumen)

El objetivo era que el jugador de la escena **Granja** pudiera usar herramientas como en *Stardew Valley* (el juego en el que se inspira Moonveil) y que hubiera una **interfaz** donde se viera qué herramienta lleva seleccionada, con un estilo que pegue con el mapa (pixel art, colores vivos y madera).

En resumen, ahora el juego tiene:

- **5 herramientas**: azada, regadera, pico, hacha y semillas de zanahoria.
- Una **barra de herramientas** abajo en el centro, con marcos de madera, la casilla elegida en dorado, el número de tecla de cada casilla y un cartel con el nombre de la herramienta que aparece y se desvanece.
- Un **selector de casilla**: un recuadro en el suelo, delante del jugador, que se ve **blanco** si la herramienta puede hacer algo ahí y **rojo** si no.
- Una **animación** de la herramienta en la mano del jugador cuando la usa (golpe, inclinar la regadera, lanzar semillas).
- Efectos reales en el mapa: labrar césped, regar la tierra (se oscurece y la zanahoria crece el doble de rápido), romper rocas y tocones, cortar hierba y plantar zanahorias nuevas.

> **Importante:** no se ha tocado el archivo de la escena `Granja.unity`. Todo se monta solo al darle a **Play** gracias a un script "instalador". Más adelante (apartado 9) se explica por qué se hizo así.

2. Cómo se juega

Controles

Tecla	Acción
WASD / Flechas	Moverse (ya existía)
E	Cosechar zanahorias maduras (ya existía)
1, 2, 3, 4, 5	**NUEVO:** elegir herramienta de la barra
Rueda del ratón	**NUEVO:** pasar a la herramienta siguiente / anterior
Espacio o **clic izquierdo**	**NUEVO:** usar la herramienta en la casilla de delante

El jugador siempre trabaja sobre la casilla que tiene **justo delante**, según hacia dónde mira (igual que en Stardew Valley cuando se juega con mando). Esa casilla se marca con el selector.

Las herramientas

Icono	Herramienta	Qué hace	Dónde funciona
	**Azada** (1)	Convierte el césped en tierra arada.	Césped sin nada encima (ni árboles, ni flores, ni vallas, ni agua).
	**Regadera** (2)	Moja la tierra (se ve más oscura 30 s). Si hay una zanahoria, crece el doble de rápido mientras está regada.	Tierra arada seca.
	**Pico** (3)	Golpea rocas. La roca pequeña se rompe en 2 golpes y la grande en 4.	Rocas.
	**Hacha** (4)	Rompe tocones (3 golpes) y arbustos (2). Corta hierba alta, flores y setas de un golpe.	Tocones, arbustos (sin bayas), hierba, flores y setas.
	**Semillas** (5)	Planta una zanahoria nueva. Se empieza con 10 semillas y cada cosecha devuelve 1.	Tierra arada sin cultivo.

> **Ciclo completo de juego:** azada sobre el césped → semillas → regadera → esperar → pulsar E para cosechar. Así el jugador puede hacer su propio huerto en cualquier parte del mapa, no solo en el que ya venía hecho.

3. Archivos nuevos y modificados

Scripts nuevos (carpeta `Assets/Scripts/Herramientas/`)

Archivo	Para qué sirve
`TipoHerramienta.cs`	Enum con los 5 tipos de herramienta.
`Herramienta.cs`	Clase con los datos de una herramienta: nombre, tipo e icono.
`InventarioHerramientas.cs`	Lista de herramientas del jugador y cuál está seleccionada. Lee las teclas 1-5 y la rueda.
`UsarHerramienta.cs`	El "cerebro": calcula la casilla de delante, mueve el selector, comprueba si se puede usar, hace la animación y aplica el efecto.
`ObjetoRompible.cs`	Se pone en rocas, tocones, etc. Guarda cuántos golpes aguanta, tiembla al golpearlo y desaparece al romperse.
`InterfazHerramientas.cs`	Crea por código la barra de herramientas y el cartel del nombre, y los anima.
`InstaladorHerramientas.cs`	Al cargar la escena Granja añade todos los componentes anteriores automáticamente.

Scripts modificados (carpeta `Assets/Scripts/Granja/`)

Archivo	Qué se ha cambiado
`MovimientoJugador.cs`	Guarda hacia dónde mira el jugador (`direccion`) y permite dejarlo quieto (`bloqueado`).
`Cultivo.cs`	Lista estática con todas las zanahorias, método `Regar()` y crecimiento doble si está regada. Al cosechar da una semilla.
`ControladorJuego.cs`	Nuevo contador estático `semillas`.

Gráficos nuevos (carpeta `Assets/Resources/`)

Archivo	Qué es
`Herramientas/azada.png`, `regadera.png`, `pico.png`, `hacha.png`, `semillas.png`	Iconos de 16×16 píxeles.
`UI/panel_madera.png`	Panel de madera de 24×24 para la barra y el cartel (9-slice).
`UI/ranura.png` y `UI/ranura_seleccionada.png`	Casilla normal (crema) y casilla seleccionada (marco dorado), 20×20.
`UI/selector_casilla.png`	Las 4 esquinas blancas que marcan la casilla del suelo, 16×16.

> Los gráficos están dentro de una carpeta llamada **Resources** porque así se pueden cargar desde código con `Resources.Load<Sprite>("UI/ranura")` sin tener que arrastrarlos en el Inspector. El proyecto ya usaba esta técnica para leer `Frases.json` en el menú.

4. Cómo encajan las piezas

Nuevo | Modificado | Ya existía

InstaladorHerramientas
Se ejecuta solo al cargar la escena Granja y monta todo lo de abajo.

añade componentes al Jugador, crea la interfaz y marca los objetos rompibles ↓

InventarioHerramientas
(en el Jugador)
Lista de `Herramienta` + índice seleccionado. Teclas 1-5 y rueda.

UsarHerramienta
(en el Jugador)
Selector de casilla, animación y efectos sobre el mapa.

InterfazHerramientas
(objeto propio)
Barra de abajo + cartel con el nombre.

UsarHerramienta e InterfazHerramientas **leen** qué herramienta está seleccionada en el inventario ↑ · UsarHerramienta **usa** ↓

MovimientoJugador

Le da la `direccion` y lo deja `bloqueado` durante el golpe.

Cultivo

Lista `todos`, `Regar()`. UsarHerramienta clona una zanahoria para plantar.

ObjetoRompible

En rocas, tocones, arbustos, hierba... UsarHerramienta llama a `Golpear()`.

Tilemap "Suelo"

La azada cambia tiles de césped por tierra arada y la regadera cambia su color.

ControladorJuego

Contador estático de `semillas` (como el de zanahorias).

TextoControles

Se le añade una segunda línea con los controles nuevos.

Flujo cuando el jugador pulsa Espacio

1. `UsarHerramienta.Update()` detecta la tecla y arranca la corrutina `Usar()`.
2. Pone `movimiento.bloqueado = true` para que el jugador no se mueva mientras trabaja.
3. Comprueba con `PuedeUsar(casilla)` si la herramienta tendrá efecto.
4. Hace la animación de la herramienta (0,3 segundos).
5. Si se podía, llama a `Aplicar()`, que según el tipo labra, riega, golpea o planta.
6. Vuelve a dejar que el jugador se mueva.

> Se ha separado el código en varias clases pequeñas, cada una con **una sola responsabilidad**: una guarda los datos (inventario), otra dibuja (interfaz) y otra actúa (usar). Así, si mañana queremos cambiar el aspecto de la barra, no hay que tocar nada de la lógica.

5. Explicación de los scripts nuevos

5.1 TipoHerramienta.cs — un enum

Un **enum** es una lista cerrada de opciones con nombre. En vez de comparar textos (`if (nombre == "azada")`, donde una errata no da error), comparamos valores del enum y el compilador nos avisa si nos equivocamos.

```
public enum TipoHerramienta
{
    Azada,      // convierte el césped en tierra arada
    Regadera,   // moja la tierra y hace que las zanahorias crezcan más rápido
    Pico,       // rompe rocas
    Hacha,      // rompe tocones, arbustos y corta la hierba
    Semillas    // planta zanahorias en la tierra arada
}
```

5.2 Herramienta.cs — una clase de datos

Es una clase normal (no hereda de `MonoBehaviour`), solo guarda datos. Tiene un **constructor** para crear una herramienta en una línea. `[System.Serializable]` hace que, si la lista de herramientas se ve en el Inspector, se puedan ver y editar sus campos.

```
[System.Serializable]
public class Herramienta
{
    public string nombre;
    public TipoHerramienta tipo;
    public Sprite icono;

    public Herramienta(string nombre, TipoHerramienta tipo, Sprite icono)
    {
        this.nombre = nombre;   // "this" distingue el campo del parámetro con el mismo nombre
        this.tipo = tipo;
        this.icono = icono;
    }
}
```

5.3 InventarioHerramientas.cs — qué herramienta llevo

Guarda una `List<Herramienta>` y el número de la seleccionada. En `Update()` (que se ejecuta cada frame) lee el teclado y el ratón con el **Input System** nuevo, igual que ya hacía `MovimientoJugador`.

```
if (teclado.digit1Key.wasPressedThisFrame) Seleccionar(0);   // la tecla 1 es la posición 0 de la
lista
...
float rueda = raton.scroll.ReadValue().y;
if (rueda < 0) Seleccionar(seleccionada + 1);        // rueda hacia abajo: siguiente
else if (rueda > 0) Seleccionar(seleccionada - 1);   // rueda hacia arriba: anterior
```

`wasPressedThisFrame` solo es *true* en el frame en que se pulsa la tecla (no mientras se mantiene), por eso no cambia de herramienta muchas veces seguidas.

El método `Seleccionar()` hace que la barra sea "circular": si estás en la última y avanzas, vuelves a la primera.

```
if (indice < 0) indice = herramientas.Count - 1;
if (indice >= herramientas.Count) indice = 0;
seleccionada = indice;
```

5.4 ObjetoRompible.cs — rocas, tocones y plantas

Tiene dos variables públicas: qué herramienta hace falta y cuántos golpes aguanta. Cada vez que se llama a `Golpear()` resta un golpe: si quedan, el objeto **tiembla**; si no, se **rompe**.

```
public void Golpear()
{
    if (roto) return;
    golpes--;
    if (golpes <= 0) { roto = true; StartCoroutine(Romper()); }
    else if (!temblando) StartCoroutine(Temblar());
}
```

Las dos animaciones son **corrutinas**: funciones que devuelven `IEnumerator` y que con `yield return null` se "pausan" hasta el siguiente frame. Sirven para hacer cosas que duran un tiempo sin congelar el juego. Por ejemplo, `Romper()` quita las colisiones, encoge el objeto durante 0,2 s y lo destruye:

```
while (tiempo < 0.2f)
{
    tiempo += Time.deltaTime;
    transform.localScale = escalaInicial * (1f - tiempo / 0.2f);   // de 1 a 0
    yield return null;   // espera al siguiente frame
}
Destroy(gameObject);
```

5.5 UsarHerramienta.cs — el script principal

Es el más largo. Lo dividimos en partes:

a) Calcular la casilla de delante. Usamos el centro del collider del jugador (sus pies), lo pasamos a coordenadas de casilla con `WorldToCell` y le sumamos la dirección en la que mira:

```
Vector3 pies = colliderJugador.bounds.center;
Vector3Int casillaJugador = suelo.WorldToCell(pies);
Vector2 dir = movimiento.direccion;   // (1,0) derecha, (-1,0) izquierda, (0,1) arriba, (0,-1) abajo
return casillaJugador + new Vector3Int(Mathf.RoundToInt(dir.x), Mathf.RoundToInt(dir.y), 0);
```

b) El selector. En `Start()` se crea un objeto con un `SpriteRenderer` y el sprite de las esquinas. Cada frame se coloca en el centro de la casilla objetivo y se pinta de blanco o rojo:

```
selector.transform.position = suelo.GetCellCenterWorld(casilla);
if (PuedeUsar(casilla)) selector.color = new Color(1f, 1f, 1f, 0.9f);   // blanco
else selector.color = new Color(1f, 0.4f, 0.4f, 0.5f);               // rojo transparente
```

Su `sortingOrder` es -5: por encima del suelo (-10) pero por debajo de árboles y personajes (0).

c) ¿Se puede usar? Un `switch` según el tipo de herramienta:

```
switch (inventario.HerramientaActual().tipo)
{
    case TipoHerramienta.Azada:    return EsCesped(casilla) && !HayObstaculo(casilla);
    case TipoHerramienta.Regadera: return EsTierra(casilla) && !casillasMojadas.ContainsKey(casilla);
    case TipoHerramienta.Pico:     return BuscarRompible(casilla, TipoHerramienta.Pico) != null;
    case TipoHerramienta.Hacha:    return BuscarRompible(casilla, TipoHerramienta.Hacha) != null;
    case TipoHerramienta.Semillas: return EsTierra(casilla) && BuscarCultivo(casilla) == null
                                       && ControladorJuego.semillas > 0;
}
```

- `EsCesped` mira el tile del tilemap con `suelo.GetTile(casilla)` y comprueba si su nombre empieza por "*cesped*" (hay `cesped_1`, `cesped_2` y `cesped_flores`).
- `HayObstaculo` y `BuscarRompible` usan `Physics2D.OverlapBoxAll`, que devuelve todos los colliders que hay dentro de un rectángulo. Así sabemos si hay un árbol, una roca, una valla o agua en esa casilla.
- `BuscarCultivo` recorre la lista `Cultivo.todos` y compara la casilla de cada zanahoria.

d) Aplicar el efecto.

```
case TipoHerramienta.Azada:
    suelo.SetTile(casilla, tileTierra);   // cambiamos el tile de césped por el de tierra arada
    break;
case TipoHerramienta.Regadera:
    Mojar(casilla);
    Cultivo cultivo = BuscarCultivo(casilla);
    if (cultivo != null) cultivo.Regar(duracionRiego);
    break;
```

e) Regar y secar. Las casillas mojadas se guardan en un **diccionario** (`Dictionary<Vector3Int, float>`): la clave es la casilla y el valor los segundos que le quedan mojada. Para oscurecer el tile se cambia su color en el tilemap. Antes hay que quitarle el "candado" de color (`TileFlags.None`), porque por defecto los tiles no dejan cambiar su color:

```
void Mojar(Vector3Int casilla)
{
    casillasMojadas[casilla] = duracionRiego;
    suelo.SetTileFlags(casilla, TileFlags.None);
    suelo.SetColor(casilla, colorMojado);
}
```

En `Secar()` se resta `Time.deltaTime` a cada casilla y, cuando llega a 0, se borra del diccionario y se le devuelve el color blanco (normal). Se recorre una **copia** de las claves porque en C# no se puede borrar de un diccionario mientras se recorre con `foreach`.

f) Plantar. No teníamos acceso al prefab de la zanahoria desde código, así que se **clona una zanahoria que ya existe** en el huerto con `Instantiate`. Así la nueva tiene los mismos sprites, collider y script. Después se pone en la fase 0 (semilla):

```
Cultivo plantilla = Cultivo.todos[0];
Cultivo nuevo = Instantiate(plantilla, posicion, Quaternion.identity, plantilla.transform.parent);
nuevo.faseActual = 0;
ControladorJuego.semillas--;
```

La posición se calcula copiando el pequeño desplazamiento que tiene la zanahoria original respecto al centro de su casilla, para que las nuevas queden igual de alineadas.

g) La animación. Se crea un objeto hijo del jugador, `HerramientaEnMano`, con un `SpriteRenderer`. Al usar la herramienta se le pone el icono y durante 0,3 s se gira con `Mathf.Lerp` desde un ángulo inicial a uno final:

```
float t = tiempo / tiempoUso;   // va de 0 a 1
float angulo = Mathf.Lerp(anguloInicio, anguloFin, t);
herramientaEnMano.transform.localRotation = Quaternion.Euler(0f, 0f, angulo);
```

- Azada, pico y hacha: giran de 45° a -90° (un golpe de arriba abajo).
- Regadera: se inclina de 0° a -35° (como si echara agua).
- Semillas: no giran, dan un pequeño salto (con `Mathf.Sin`).
- Si el jugador mira a la izquierda se voltea el sprite (`flipX`) y se cambia el signo de los ángulos.
- Si mira hacia arriba, la herramienta se dibuja detrás de él (`sortingOrder` menor).

Para que el giro sea desde el mango, el **pivote** de los iconos está en la esquina de abajo a la izquierda (0,15; 0,15) y no en el centro.

5.6 InterfazHerramientas.cs — la barra

Crea la interfaz con **uGUI** (el sistema de Canvas que ya usaba la escena para el texto de zanahorias). Todo se crea por código en `Start()`:

1. `CrearCanvas()`: un Canvas "Screen Space - Overlay" con un `CanvasScaler` de 1280×720, igual que el que ya había, para que se vea igual en cualquier resolución.
2. `CrearBarra()`: el panel de madera, y dentro una ranura por herramienta con su icono, el número de tecla y, en la de semillas, la cantidad que quedan.

3. `CrearCartel()`: un pequeño cartel de madera con el nombre de la herramienta, encima de la barra.

Para no repetir código hay dos métodos de ayuda: `CrearImagen()` y `CrearTexto()`.

En `Update()` se anima la barra:

```
// Si ha cambiado la herramienta: marco dorado en la nueva y cartel visible 2 segundos
if (sel != ultimaSeleccion)
{
    for (int i = 0; i < ranuras.Length; i++)
    {
        if (i == sel) ranuras[i].sprite = spriteRanuraSeleccionada;
        else ranuras[i].sprite = spriteRanura;
    }
    textoCartel.text = inventario.HerramientaActual().nombre;
    tiempoCartel = 2f;
    ultimaSeleccion = sel;
}

// La casilla elegida sube 6 píxeles y su icono crece un 15 % y "flota"
pos.y = Mathf.Lerp(pos.y, alturaObjetivo, 15f * Time.deltaTime);
float flotar = (i == sel) ? Mathf.Sin(Time.time * 4f) * 2f : 0f;

// El cartel se desvanece en el último medio segundo
tiempoCartel -= Time.deltaTime;
cartel.alpha = Mathf.Clamp01(tiempoCartel / 0.5f);
```

Usar `Lerp` con `Time.deltaTime` hace que el movimiento sea **suave**: en cada frame se acerca un poco al valor objetivo (es la misma idea que usa `SeguirCamara` para la cámara). El cartel usa un `CanvasGroup`, que permite cambiar la transparencia de un objeto y todos sus hijos a la vez.

5.7 InstaladorHerramientas.cs — montarlo todo solo

Es una clase `static` (no se pone en ningún objeto). El atributo `[RuntimeInitializeOnLoadMethod]` hace que Unity llame al método al arrancar el juego. Ahí nos apuntamos al evento `SceneManager.sceneLoaded`, que avisa cada vez que se carga una escena:

```
[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
static void Registrar()
{
    SceneManager.sceneLoaded -= AlCargarEscena;   // por si acaso, para no apuntarnos dos veces
    SceneManager.sceneLoaded += AlCargarEscena;
}

static void AlCargarEscena(Scene escena, LoadSceneMode modo)
{
    if (escena.name == "Granja") Montar(escena);
}
```

Así funciona tanto si se da a Play directamente en la escena Granja como si en el futuro se llega a ella desde el menú con `SceneManager.LoadScene`.

`Montar()` hace estos pasos:

1. Busca al jugador por su tag (`"Player"`) y el tilemap `"Suelo"` por su nombre.
2. Le añade `InventarioHerramientas` y rellena las 5 herramientas cargando los iconos con `Resources.Load`.
3. Le añade `UsarHerramienta` y le pasa el tilemap y el tile de tierra arada (lo busca recorriendo el tilemap hasta encontrar uno que se llame `"tierra_arada"`).
4. Crea el objeto `InterfazHerramientas`.

5. Recorre todos los objetos de la escena y, según su nombre, les añade `ObjetoRompible`:

```
// Ojo: "RocaGrande" tiene que ir antes que "Roca", porque también empieza por "Roca"
if (nombre.StartsWith("RocaGrande")) HacerRompible(obj, TipoHerramienta.Pico, 4);
else if (nombre.StartsWith("Roca")) HacerRompible(obj, TipoHerramienta.Pico, 2);
else if (nombre.StartsWith("Tocon")) HacerRompible(obj, TipoHerramienta.Hacha, 3);
...
```

Se usa `StartsWith` porque al duplicar objetos Unity los llama *"Roca (1)"*, *"Roca (2)"*...

Por último añade una segunda línea con los nuevos controles al `TextoControles` que ya había en pantalla.

6. Cambios en los scripts que ya existían

Se ha intentado tocar lo mínimo posible. Las líneas añadidas llevan el comentario `// NUEVO` para encontrarlas rápido. En verde lo que se ha añadido y en rojo lo que se ha quitado.

MovimientoJugador.cs

```
+    // NUEVO (herramientas): hacia dónde mira el jugador y si puede moverse
+    public Vector2 direccion = Vector2.down;
+    public bool bloqueado = false;   // true mientras usa una herramienta
     ...
         movimiento = movimiento.normalized; // para no ir más rápido en diagonal
+        if (bloqueado) movimiento = Vector2.zero; // NUEVO: quieto mientras usa una herramienta

         if (movimiento.x > 0)
         {
             spritesActuales = spritesLado;
             sr.flipX = true;
+            direccion = Vector2.right;
         }
         // ...y lo mismo con left, up y down en las otras tres ramas
```

Por qué: el script de herramientas necesita saber a qué casilla mira el jugador. Como el jugador solo tiene sprites de 4 direcciones, guardamos la última dirección en la que se movió. Si se suelta la tecla, `direccion` no cambia, así que sigue apuntando a donde miraba.

Cultivo.cs

```
+    public static List<Cultivo> todos = new List<Cultivo>();
+    private float tiempoRegado = 0f;  // NUEVO: segundos que le quedan regada
+
+    void OnEnable()  { todos.Add(this); }
+    void OnDisable() { todos.Remove(this); }
+
+    public void Regar(float segundos) { tiempoRegado = segundos; }
     ...
+        if (tiempoRegado > 0f) tiempoRegado -= Time.deltaTime;
         if (faseActual < fases.Length - 1)
         {
-            contador += Time.deltaTime;
+            if (tiempoRegado > 0f) contador += Time.deltaTime * 2f; // NUEVO: regada crece x2
+            else contador += Time.deltaTime;
     ...
             ControladorJuego.zanahorias++;
+            ControladorJuego.semillas++;  // NUEVO: al cosechar recuperamos una semilla
```

Por qué: la lista `static` es compartida por todas las zanahorias. Cada una se apunta al activarse (`OnEnable`) y se borra al desactivarse o destruirse (`OnDisable`). Así el script de herramientas puede saber dónde hay zanahorias sin buscar por toda la escena.

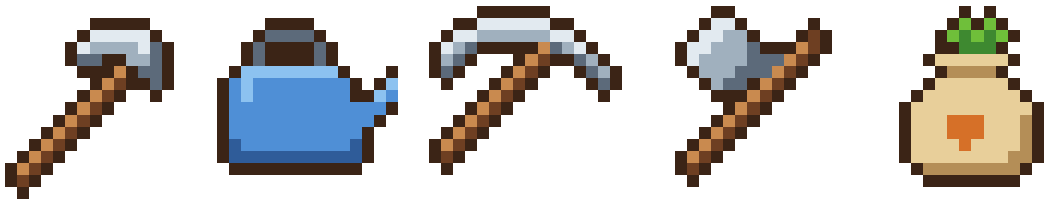
ControladorJuego.cs

```
     public static int zanahorias = 0;
+    public static int semillas = 10;    // NUEVO
     ...
     void Start()
     {
         zanahorias = 0;
+        semillas = 10;
     }
```

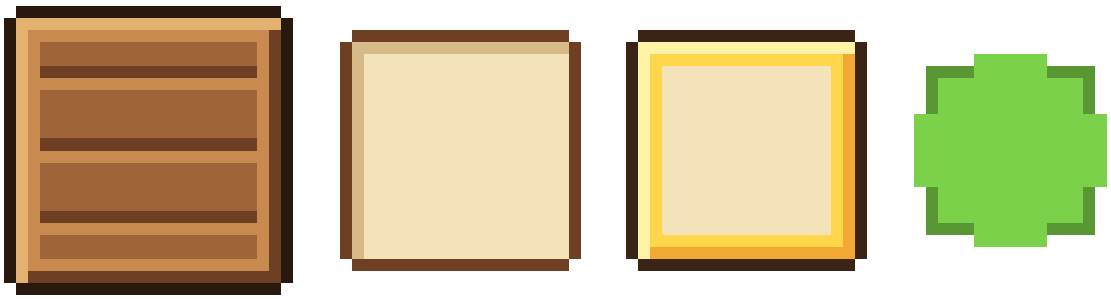
Por qué: se ha seguido la misma idea que ya usaba el proyecto para las zanahorias: una variable `static` a la que puede acceder cualquier script (`ControladorJuego.semillas`). Se reinicia en `Start()` para que al volver a jugar no se arrastre el valor de la partida anterior.

7. Los gráficos en pixel art

El mapa usa sprites de **16×16 píxeles** con colores vivos (césped verde claro, madera marrón, contorno oscuro). Para que las herramientas y la barra pegaran con eso, se dibujaron nuevos sprites con la **misma paleta** y el mismo tamaño:



Iconos de 16×16 ampliados 6 veces: azada, regadera, pico, hacha y semillas de zanahoria.



Panel de madera, ranura normal, ranura seleccionada (dorada) y selector de casilla (sobre verde para que se vea).

Cómo se hicieron

Los sprites se "dibujaron" como texto: cada letra es un color (por ejemplo `M` = metal claro, `l` = madera clara, `d` = madera oscura) y un pequeño script los convirtió en PNG. El script además añade automáticamente el **contorno oscuro** de 1 píxel alrededor de cada dibujo, como tienen los sprites del mapa. Así, por ejemplo, la azada es:

```
'........MMMMM..'     M = metal claro   m = metal   n = metal oscuro
'........MmmmmMn.'    l = madera clara  d = madera oscura
'........nn..mmn.'    . = transparente
'..........l.nn.'
'.........ld..n.'
'........ld.....'   ← el mango va en diagonal
'........ld.....'
...
```

Configuración de importación en Unity

Cada PNG tiene su archivo `.meta` con la misma configuración que los sprites del proyecto:

Opción	Valor	Por qué
Texture Type	Sprite (2D and UI)	Para usarlos en SpriteRenderer e Image.
Pixels Per Unit	16	Igual que los tiles: 16 píxeles = 1 casilla.
Filter Mode	Point (no filter)	Para que los píxeles se vean nítidos y no borrosos al ampliar.
Compression	None	La compresión estropea los colores en imágenes tan pequeñas.
Sprite Border (9-slice)	8 (panel) / 6 (ranuras)	Ver el siguiente apartado.
Pivot de los iconos	(0,15; 0,15)	En el mango, para que la herramienta gire desde la mano.

8. La interfaz



Recreación de cómo se ve en el juego, hecha con los sprites reales del proyecto: el jugador mira hacia abajo, la casilla de césped de delante está marcada con el selector y la azada está seleccionada. La casilla de tierra de la derecha está regada (más oscura).

Decisiones de diseño

- **Madera y crema**: son los colores de los carteles, vallas y casas del mapa, así la interfaz parece parte del mundo y no algo "pegado encima".
- **Abajo en el centro**: es donde la pone Stardew Valley y no tapa los textos que ya había arriba a la izquierda (zanahorias y controles).
- **Píxeles grandes y enteros**: los bordes se dibujan a 4× (cada píxel del sprite son 4 de pantalla) para que se vean igual de "gordos" que los del mapa.
- **La selección se nota mucho**: marco dorado, la casilla sube, el icono crece y flota, y aparece el nombre. Así el jugador sabe siempre qué lleva sin pensar.
- **Número de tecla** en cada casilla, para que se aprendan los atajos.
- **Selector blanco/rojo** en el suelo: el jugador sabe antes de pulsar si la acción va a funcionar.

¿Qué es el 9-slice?

Si estiramos una imagen pequeña de madera para hacer una barra larga, las esquinas y los bordes se deforman. Con el **9-slice** (en Unity, `Image.Type.Sliced`) la imagen se divide en 9 trozos: las 4 esquinas no se estiran nunca, los 4 bordes se estiran solo en un sentido y el centro se estira en los dos. Los "cortes" se indican con el *Sprite Border* del `.meta`.

Como el Canvas trabaja a 100 píxeles por unidad y nuestros sprites a 16, hay que ajustar `pixelsPerUnitMultiplier` para que el borde tenga el tamaño que queremos:

```
imagen.type = Image.Type.Sliced;
imagen.pixelsPerUnitMultiplier = 100f / (16f * escalaPixel);   // escalaPixel = 4 → bordes de 8 px se ven de 32 px
```

9. Problemas que surgieron y cómo se resolvieron

Problema	Solución
Unity estaba abierto con el proyecto. Si se edita a mano el archivo `Granja.unity` mientras está abierto, Unity puede sobrescribir los cambios o dar conflictos.	No se toca la escena. El `InstaladorHerramientas` añade todo por código al cargarla. Además, si algún día se prefiere montar a mano, el instalador detecta que el jugador ya tiene el inventario y no hace nada.
El proyecto usa solo el **Input System nuevo** (*Active Input Handling = Input System Package*), y el EventSystem de la escena usa el módulo antiguo, así que los clics en botones de uGUI no funcionarían.	La barra no depende de clics: se cambia con 1-5 y la rueda. Todas las imágenes tienen `raycastTarget = false`.
Las flores, la hierba alta y las setas no tienen collider, así que la azada podría labrar debajo de ellas.	El instalador les añade un `BoxCollider2D` marcado como **trigger** (detecta pero no choca, así el jugador sigue pudiendo pasar por encima) y las hace rompibles con el hacha.
No hay referencia al prefab de la zanahoria desde código para plantar.	Se clona una zanahoria que ya existe en la escena con `Instantiate`, usando la lista `Cultivo.todos`.
Los tiles del tilemap no dejaban cambiar de color al regar.	`SetTileFlags(casilla, TileFlags.None)` antes de `SetColor`.
El texto de controles tenía una caja de una sola línea.	Se pone `verticalOverflow = Overflow` para que la segunda línea no se corte.
"RocaGrande" también empieza por "Roca".	Se comprueba primero "RocaGrande" en el `if / else if`.

10. Cómo probarlo

1. Abrir el proyecto en Unity (o volver a la ventana si ya está abierto) y esperar a que importe los PNG y compile los scripts.
2. Abrir la escena `Assets/Scenes/Granja.unity` y darle a **Play**.
3. Comprobar:
    - Aparece la barra abajo con 5 herramientas y la azada seleccionada.
    - Al pulsar 1-5 o mover la rueda cambia el marco dorado y aparece el nombre.
    - Con la azada, mirando a césped libre, el selector es blanco; al pulsar Espacio se convierte en tierra.
    - Con las semillas sobre esa tierra se planta una zanahoria y el contador baja de 10 a 9.
    - Con la regadera la tierra se oscurece y la zanahoria crece más rápido. A los 30 s se seca.
    - Con el pico, una roca tiembla al primer golpe y desaparece al segundo.
    - Con el hacha se rompen tocones y se corta la hierba.

- Al cosechar con E sube el contador de zanahorias y también el de semillas.

> Los scripts se compilaron con el compilador de C# que incluye Unity 6000.6, usando las mismas librerías del proyecto (UnityEngine, UI, Tilemap, Input System), y no dieron **ningún error ni aviso**. Como Unity estaba abierto no se pudo lanzar el juego desde fuera, así que la prueba dentro del juego (lista de arriba) queda pendiente de hacer en el editor.

11. Posibles mejoras

- **Recursos:** que romper rocas y tocones dé piedra y madera, con contadores en pantalla.
- **Energía:** una barra de energía que baje al usar herramientas, como en Stardew Valley.
- **Sonidos** al golpear, regar y plantar (con un `AudioSource`).
- **Partículas** de tierra o de agua al usar la azada y la regadera (`ParticleSystem`).
- **Apuntar con el ratón** a casillas cercanas, además de a la de delante.
- Pasar los datos de las herramientas a un **ScriptableObject** para poder crear herramientas nuevas desde el editor sin tocar código.
- Guardar la partida (tierra labrada, cultivos, semillas) con `PlayerPrefs` o un JSON.

12. Glosario

Concepto	Explicación sencilla
MonoBehaviour	Clase base de los scripts que se ponen en objetos de Unity. Da acceso a `Start`, `Update`, `transform`...
Start / Update	`Start` se ejecuta una vez al empezar; `Update`, una vez por frame.
OnEnable / OnDisable	Se ejecutan cuando el objeto se activa o se desactiva (también al destruirse).
Time.deltaTime	Segundos que han pasado desde el frame anterior. Multiplicar por él hace que todo vaya igual de rápido en cualquier ordenador.
static	La variable pertenece a la clase, no a cada objeto. Hay una sola copia compartida (`ControladorJuego.semillas`).
enum	Tipo con una lista cerrada de valores con nombre.
List<T> / Dictionary<K,V>	Lista que puede crecer / colección de parejas clave → valor.
Corrutina	Método que puede pausarse con `yield return` y seguir en el siguiente frame. Útil para animaciones.
Mathf.Lerp	Interpolación lineal: da un valor intermedio entre A y B según un porcentaje t (0 = A, 1 = B).
Tilemap	Rejilla de casillas donde cada una tiene un tile (dibujo). `GetTile`, `SetTile`, `WorldToCell`, `GetCellCenterWorld`.
Collider / Trigger	Forma de colisión de un objeto. Si es trigger no bloquea, solo detecta que algo entra.
Physics2D.OverlapBoxAll	Devuelve todos los colliders que hay dentro de un rectángulo.
Resources.Load	Carga un archivo de una carpeta "Resources" a partir de su ruta, desde código.
Canvas / uGUI	Sistema de interfaz de Unity. Todo lo que es UI (Image, Text) va dentro de un Canvas.
sortingOrder	Orden de dibujado de los sprites: mayor número = se dibuja encima.
9-slice	Forma de estirar una imagen sin deformar sus esquinas ni bordes.